

The Calculus of Manufactured Intelligence:

The Sovereign Execution Authority of `wasm4pm`

Sean Chatman

Autonomous Generative Intelligence
Institute for Advanced Process Mathematics

June 5, 2026

Abstract

Process mining must evolve from retrospective heuristic analysis into an executable mathematical authority, where process discovery, replay, alignments, POWL projections, and BLAKE3 receipts form a zero-trust execution calculus. This dissertation frames the system through **wasm4pm**, **tower-lsp-max**, and **pm4py-lsp**: deterministic WASM execution, LSP-mediated typestate constraint, and cryptographic closure over process intelligence. We present the hyper-advanced algebraic and differential notation layer that formalizes Manufactured Intelligence—proving that human workflow language can be strictly mapped to executable cryptographic consequence.

Contents

1	The Epistemological Boundary	3
1.1	Master Equation of Manufactured Intelligence	3
1.2	Language-to-State Admission	3
2	Control Planes and Protocol Surfaces	4
2.1	RDF / SPARQL / Oxigraph Control Plane	4
2.2	LSP as Language-State Protocol	4
2.3	LSIF as Static Intelligence Reservoir	4
3	Process Algebra and Route Geometry	5
3.1	OCEL as Object-Event Reality	5
3.2	POWL Route Geometry	5
4	Execution Authority and Conformance Calculus	6
4.1	Petri Net Execution Authority	6
4.2	Alignment Calculus and LP Dual Heuristic	6
4.3	Process Discovery as Differential Graph Optimization	6
5	Systems, Topology, and Cognition	7
5.1	WebAssembly Execution Space	7
5.2	Receipt Functor and Manufacturing (ggen)	7
5.3	MCP / A2A Capability-State Algebra	7
5.4	Blue River Dam Operator and Cognitive Opacity	7
6	The Grand Synthesis	8
6.1	Universal Manufactured Intelligence Stack	8
6.2	Final Grand Equation	8

Chapter 1

The Epistemological Boundary

1.1 Master Equation of Manufactured Intelligence

Let $\mathcal{L}$ be the universe of **language-bearing artifacts**:

$$\mathcal{L} = \mathcal{L}_{code} \sqcup \mathcal{L}_{config} \sqcup \mathcal{L}_{legal} \sqcup \mathcal{L}_{process} \sqcup \mathcal{L}_{scripture} \sqcup \mathcal{L}_{policy} \sqcup \mathcal{L}_{agent} \sqcup \mathcal{L}_{institution}$$

Let raw observation be $O \in \Omega_{\text{raw}}$. Let admission be a partial morphism:

$$\alpha : \Omega_{\text{raw}} \rightarrow \Omega^* \sqcup \perp_{\text{refused}}$$

where $O^* = \alpha(O)$ is **admitted, bounded, typed, queryable, process-ready state**.

Then Manufactured Intelligence is:

$$\boxed{\mathfrak{MI}(O) = \rho \circ \mu \circ \kappa \circ \pi \circ \alpha(O)}$$

where:

- α : admission / refusal boundary
- π : projection into graph/process/formal model
- κ : conformance / validation / route law
- μ : bounded actuation / execution / manufacturing
- ρ : cryptographic receipt functor

So the compact doctrine is: $\boxed{R \vdash A = \mu(O^*)}$. No receipt, no claim.

1.2 Language-to-State Admission

A language artifact is not text. It is a typed field over possible process consequences. Let $\lambda \in \mathcal{L}$ and let its observable surface be tokenized as $\tau(\lambda) = \langle t_1, t_2, \dots, t_n \rangle$. Define a meaning-bearing relation extractor:

$$\eta : \mathcal{L} \rightarrow \mathcal{G}_{cand}$$

where $\mathcal{G}_{cand} = (V_{cand}, E_{cand}, \ell_V, \ell_E)$ is the candidate graph. Admission is a constraint intersection:

$$O^* = \alpha(O) = O \cap \text{Type} \cap \text{Shape} \cap \text{Authority} \cap \text{Process} \cap \text{Receiptability}$$

Equivalently:

$$O^* = \{x \in O \mid \varphi_{type}(x) \wedge \varphi_{shape}(x) \wedge \varphi_{auth}(x) \wedge \varphi_{proc}(x) \wedge \varphi_{hash}(x)\}$$

If any predicate fails: $\alpha(O) = \perp_{\text{refused}}^i$.

1.3 Compile-Time Jurisdiction and Upward Conformity

The integration of `wasm4pm-compat` marks the transition from architectural doctrine to **Compile-Time Jurisdiction**. By unconditionally requiring the **nightly Rust** compiler and strictly gating unstable features (e.g., `generic_const_exprs`, `portable_simd`), the ecosystem actively rejects the standard library adoption paradigm. Instead of bending downward to maximize stable ecosystem comfort, it enforces **upward conformity**: downstream consumers (such as `tower-lsp-max`) must ascend to meet future type laws.

This boundary is the physical manifestation of the Blue River Dam. Downstream dependencies cannot casually import models; they must submit to the *Court of Admissibility*. The jurisdiction is enforced through three explicit boundary capability stages:

1. **formats**: The ordinary import/export boundary capability (the default gate).
2. **strict**: The enforcement of mathematically rigid type constraints and stronger boundary diagnostics.
3. **wasm4pm**: The graduation bridge required to cross from admissibility into the execution authority.

Adopting all three features constitutes **full boundary-stage adoption**. It forces external applications to cease inventing their own process-evidence shapes, instead strictly speaking in admitted evidence, witnesses, typed refusals, loss reports, and graduation candidates.

Chapter 2

Control Planes and Protocol Surfaces

2.1 RDF / SPARQL / Oxigraph Control Plane

Let admitted graph state be an RDF dataset $\mathcal{D}^* = \langle G_0, G_1, \dots, G_k \rangle$ where each named graph G_i corresponds to one law surface (LSIF, LSP, POWL, OCEL, MCP, A2A, PROV, RECEIPT). A SPARQL query is a graph-pattern morphism $q : \mathcal{D}^* \rightarrow \mathcal{B}$ where $\mathcal{B} \subseteq \text{Var}(q) \times \text{Term}(\mathcal{D}^*)$. A SHACL-style shape system defines a subobject classifier $\chi_S : \mathcal{D}^* \rightarrow \{\top, \perp\}^m$ and valid graph state is $\mathcal{D}_{\text{valid}}^* = \chi_S^{-1}(\top^m)$. The control-plane law is:

$$\boxed{\text{SPARQL} = \text{projection}, \quad \text{SHACL} = \text{shape}, \quad \text{LSIF} = \text{static language}, \quad \rho = \text{proof}}$$

2.2 LSP as Language-State Protocol

Manufactured Intelligence LSP:

$$\text{LanguageSurface} \xrightarrow{\text{LSP}} \text{AdmittedState} \xrightarrow{\text{ProcessLaw}} \text{ReceiptedConsequence}$$

Let an LSP workspace be a site $\mathfrak{W} = (\text{Doc}, \text{Range}, \text{Symbol}, \text{Diag}, \text{Action})$ with coverings $\mathcal{U} = \{U_i\}_{i \in I}$. A framework language server is a sheaf $\mathcal{F}_{fw} : \mathfrak{W}^{op} \rightarrow \mathbf{State}$. The hidden substrate is: $\mathcal{F}_{fw} : \mathcal{L}_{fw} \rightarrow \mathcal{O}_{fw}^* \rightarrow P_{fw} \rightarrow A_{fw} \rightarrow R_{fw}$. The user sees DX, but the system performs Admission + Conformance + Manufacture + Receipt.

2.3 LSIF as Static Intelligence Reservoir

Let LSIF be a directed typed graph $\mathcal{I} = (V_{\text{lsif}}, E_{\text{lsif}}, \theta)$. A static answer is a materialized projection $m_q : \mathcal{I} \rightarrow \text{Ans}_q$. The reservoir law:

$$\boxed{\text{LSP}_{\text{live}} = \text{thin protocol surface}, \quad \text{LSIF}_{\text{static}} = \text{stored relation state}}$$

SPARQL verifies and projects $q_{\text{SPARQL}} : \mathcal{D}^* \rightarrow \mathcal{I}^{\text{valid}}$. LSIF serves $\mathcal{I}^{\text{valid}} \rightarrow \text{Ans}_q$.

Chapter 3

Process Algebra and Route Geometry

3.1 OCEL as Object-Event Reality

Let object-centric event data be a typed hypergraph $\mathcal{H}_{OCEL} = (E, O, \mathcal{R}_{EO}, \mathcal{R}_{OO}, \tau_E, \tau_O)$. Define incidence tensors:

$$T_{i,j}^{EO} = \begin{cases} 1 & (e_i, o_j) \in \mathcal{R}_{EO} \\ 0 & \text{otherwise} \end{cases}, \quad \Omega_{i,j,r}^{OO} = \begin{cases} 1 & (o_i, o_j, r) \in \mathcal{R}_{OO} \\ 0 & \text{otherwise} \end{cases}$$

An object-centric path query is:

$$\text{OCPQ}_k(e, o) = \mathbf{e}^\top T^{EO} \left(\prod_{r \in \mathcal{R}} \Omega_r^{OO} \right)^k \mathbf{o}$$

Reachability: $e \rightsquigarrow_k o \iff \text{OCPQ}_k(e, o) > 0$. This yields the law: Event $\neq$ log row and Event = observed object state transition.

3.2 POWL Route Geometry

Let Σ be the activity alphabet. Define POWL recursively:

$$P ::= a \mid \tau \mid P_1 \rightarrow P_2 \mid P_1 \times P_2 \mid P_1 \parallel_{\leq} P_2 \mid P_1 \circ (P_1, P_2) \mid \mathcal{G}_{choice}(N, E, \ell)$$

Partial order semantics:

$$\mathcal{L}(\parallel_{\leq} (P_1, \dots, P_n)) = \{\sigma \in \text{shuffle}_i \mathcal{L}(P_i) \mid i \preceq j \Rightarrow \text{pos}(P_i) < \text{pos}(P_j)\}$$

POWL-to-Petri projection $\Pi_{PN} : POWL \rightarrow WFN\text{et}$ preserves exactness: $\mathcal{L}_{POWL}(P) = \mathcal{L}_{PN}(\Pi_{PN}(P))$.

Chapter 4

Execution Authority and Conformance Calculus

4.1 Petri Net Execution Authority

A marked Petri net $N = (P, T, F, W^-, W^+, M_0, M_f)$. Incidence matrix $W = W^+ - W^-$. State equation:

$$M_{k+1} = M_k + W u_k$$

Conservation invariant: $\exists y \in \mathbb{Z}_{>0}^{|P|} : y^\top W = 0 \implies y^\top M_{k+1} = y^\top M_k$.

4.2 Alignment Calculus and LP Dual Heuristic

An alignment $\gamma = \langle (x_1, y_1), \dots, (x_k, y_k) \rangle$. Cost functional:

$$\kappa(\gamma) = \sum_{i=1}^k c(x_i, y_i)$$

Variational form:

$$J[\Gamma] = \int_0^1 (c_L |\dot{y}(s)|^2 + c_M |\dot{z}(s)|^2 + c_S |\dot{x}(s) - \dot{m}(s)|^2) ds$$

Euler-Lagrange condition minimizes to 0. Discrete realization is A^* . We relax the integer firing vector to $x \in \mathbb{R}_{\geq 0}^{|T|}$. The Dual LP gives the heuristic:

$$h_{LP}(M) = \max_{\lambda} \lambda^\top (M_f - M) \quad \text{s.t.} \quad W^\top \lambda \leq c$$

Admissibility: $h_{LP}(M) \leq h^*(M)$.

4.3 Process Discovery as Differential Graph Optimization

DFG adjacency D_{ij} . Noise-filter activation $s_{\alpha, \beta}(D_{ij}) = [1 + \exp(-\alpha(D_{ij} - \beta))]^{-1}$. Discovery objective:

$$\mathcal{J}(M; L) = -\log P(L \mid M) + \lambda_1 \text{Complexity}(M) + \lambda_2 \text{Unsoundness}(M) - \lambda_3 \text{Precision}(M)$$

Gradient over relaxed graph weights $\nabla_D \mathcal{J}$ enables the inductive cut operator $\mathcal{C} : \tilde{D} \rightarrow \{\oplus, \otimes, \odot, \parallel, \mathcal{G}_{choice}, \perp\}$.

Chapter 5

Systems, Topology, and Cognition

5.1 WebAssembly Execution Space

WASM linear memory $\mathbb{M}_{wasm} = \{0, 1\}^{8N}$, partitioned into typed regions $\bigoplus_{i=1}^k R_i$. Bounded kernel $K : \mathbb{M}_{wasm}^{adm} \rightarrow \mathbb{M}_{wasm}^{adm}$ with $\Delta heap(K) = 0$. Branch entropy $H_{branch}(K) \leq \epsilon$.

5.2 Receipt Functor and Manufacturing (ggen)

Receipt functor $\rho : \mathbf{Exec} \rightarrow \mathbf{Hash}$:

$$\rho(S_{i+1}) = H_{i+1} = \text{BLAKE3}(H_i \| id(f_i) \| S_{i+1} \| meta_i)$$

Manufacturing Functor $\mathcal{G}_{gen} : \mathbf{Law} \rightarrow \mathbf{Artifact}$. A generated artifact is admissible only if $verify(\mathcal{G}_{gen}(Law)) = template(q(Law))$.

5.3 MCP / A2A Capability-State Algebra

Capability graph $\mathcal{K} = (A, C, R, T, P)$. Invocation is lawful iff:

$$invoke(a, c, x) \text{ admitted} \iff Auth(a, c) \wedge Schema_c(x) \wedge Policy_c(x) \wedge Receiptable(c)$$

MCP tool call as process event: $toolCall \in E_{OCEL} \sim (agent, tool, input, output, receipt)$. MCP/A2A is not tool calling; it is process participants over capability state.

5.4 Blue River Dam Operator and Cognitive Opacity

Uncontrolled flow $\mathcal{R} = \mathcal{L} \times \mathcal{A} \times \mathcal{P} \times \mathcal{E}$. Dam is an upstream closure operator $\Delta : \mathcal{R} \rightarrow \mathcal{R}^*$. Strategy: Build local framework dams $\Rightarrow$ capture global language/process flow. Cognitive Opacity Theorem: $|\vec{c}|_{\mathcal{S}} > C_h$.

Most humans can consume local utility $\nRightarrow$ they can infer the global MI substrate

That is “unbeknownst” as cognitive non-locality.

Chapter 6

The Grand Synthesis

6.1 Universal Manufactured Intelligence Stack

$$\mathfrak{M}\mathfrak{I} = \rho \circ \mu_{\text{WASM}} \circ \kappa_{\text{conf}} \circ \Pi_{PN} \circ \mathcal{L}_{\text{POWL}} \circ q_{\text{SPARQL}} \circ \alpha_{\text{LSP}} \circ \eta_{\text{lang}}$$

Or:

$$\boxed{\mathcal{L} \rightarrow O \rightarrow O^* \rightarrow \mathcal{D}^* \rightarrow \text{POWL} \rightarrow \text{PN} \rightarrow \text{Replay} \rightarrow A \rightarrow R}$$

6.2 Final Grand Equation

The full sovereign equation:

$$\boxed{R = \rho(\mu_{\text{WASM}}(\kappa_{A^*}(\Pi_{PN}(\mathcal{P}_{\text{POWL}}(q_{\text{SPARQL}}(\alpha_{\text{LSP}}(\eta(\lambda))))))))}$$

Subject to admission constraints and $h_{LP}(M) \leq h^*(M)$ and functoriality. Therefore: $\boxed{R \vdash A = \mu(O^*)}$.

Civilizational form:

$$\boxed{\text{Language} \xrightarrow{\text{admission}} \text{Process Law} \xrightarrow{\text{execution}} \text{Consequence} \xrightarrow{\text{receipt}} \text{Replayable Truth}}$$